

STUDY NOTE

Observability와 심화 주제

01 Observability

02 심화 주제

03 핸즈온

01

Observability

Signal — Log / Metric / Trace

하나의 요청이 시스템을 지나가는 동안 세 가지 다른 형태의 신호를 남깁니다. 셋은 서로 다른 질문에 답합니다.

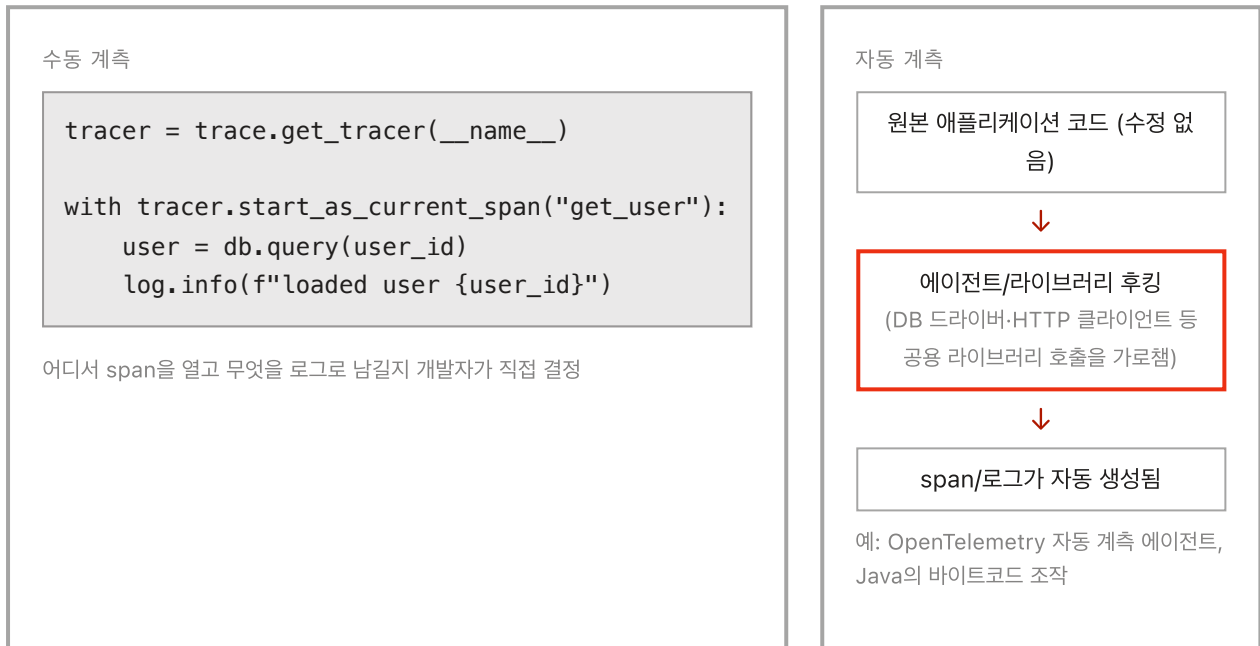


	LOG	METRIC	TRACE
형태	이벤트 텍스트/구조화 레코드	숫자 (counter, gauge, histogram)	span(요청 처리 중 한 구간의 시작·끝 시각)이 이어진 트리
저장 비용	높음 (원문 그대로)	낮음 (집계됨)	중간 (요청 단위)
대표 질문	"에러 메시지가 정확히 뭐였나"	"P99 지연시간이 올랐나"	"어느 하위 호출이 느렸나"

span은 "API 게이트웨이 처리", "DB 쿼리" 처럼 요청 처리 중 한 구간의 시작·끝 시각과 이름을 담은 기록 단위입니다. 하위 호출마다 span을 하나씩 만들고 부모-자식으로 연결하면, 요청 하나가 어디서 얼마나 걸렸는지 트리 형태로 드러납니다. 세 신호는 `trace_id`를 로그와 span에 함께 심어두면 서로 연결됩니다 — 메트릭 그래프에서 이상 구간을 찾고, 그 구간의 트레이스로 느린 span을 찾고, 그 span이 남긴 로그로 원인을 확인하는 식입니다.

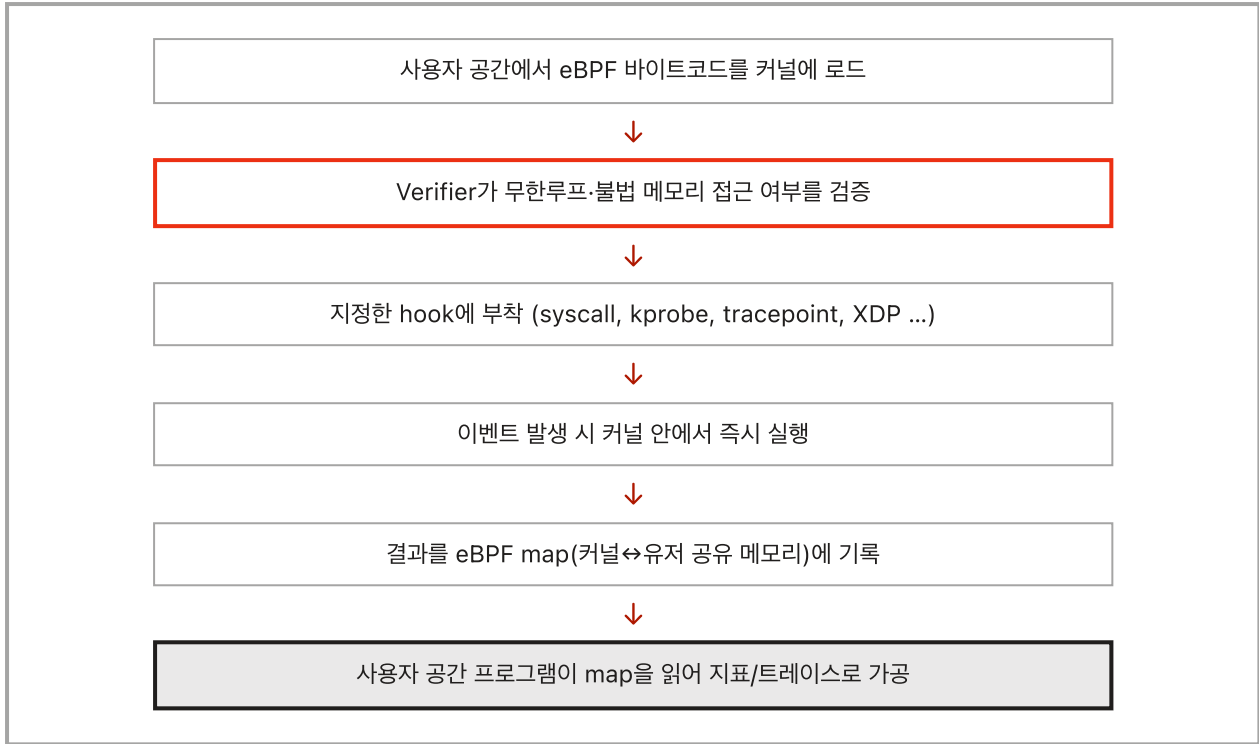
Instrumentation

계측(instrumentation)은 애플리케이션이 이 신호들을 실제로 만들어내게 하는 작업입니다. 코드를 직접 고치는 수동 방식과, 코드를 건드리지 않고 밖에서 주입하는 자동 방식이 있습니다.



eBPF

eBPF는 커널을 재컴파일하거나 커널 모듈을 올리지도 않고도, 검증된 소규모 프로그램을 커널 안의 지정된 지점(hook)에서 실행시키는 기술입니다. 애플리케이션 코드를 전혀 건드리지 않고도 시스템 콜, 네트워크 패킷, 함수 진입 시점 등을 관찰할 수 있어 계측의 세 번째 축이 됩니다.



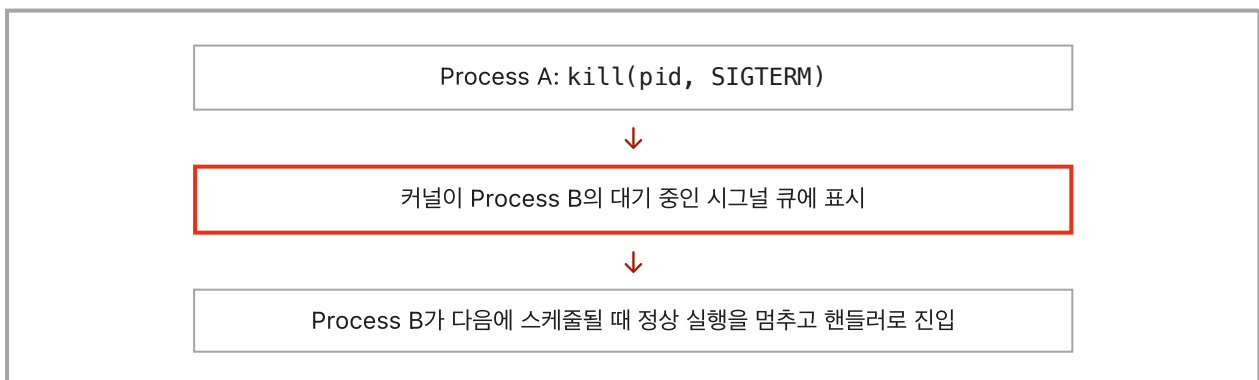
대표 사례: bpftrace(임시 조사), Cilium(네트워킹), Pixie·Parca(자동 관측성 수집) — 모두 애플리케이션 재시작이나 코드 수정 없이 동작합니다.

02

심화 주제

프로세스 간 통신 — Signal

시그널은 커널이 프로세스에게 보내는 짧은 비동기 알림입니다. 프로세스는 자신이 실행 중인 코드 흐름과 무관하게 시그널 핸들러로 즉시 끌려갑니다.



시그널	기본 동작	가로채기
SIGTERM	종료	가능 — graceful shutdown에 사용
SIGKILL	즉시 종료	불가능
SIGINT	종료	가능 (Ctrl+C)
SIGHUP	종료	데몬은 흔히 "설정 다시 읽기"로 재정의
SIGCHLD	무시	부모가 자식 종료를 감지(wait)하는 데 사용

프로세스 간 통신 — Named Pipe

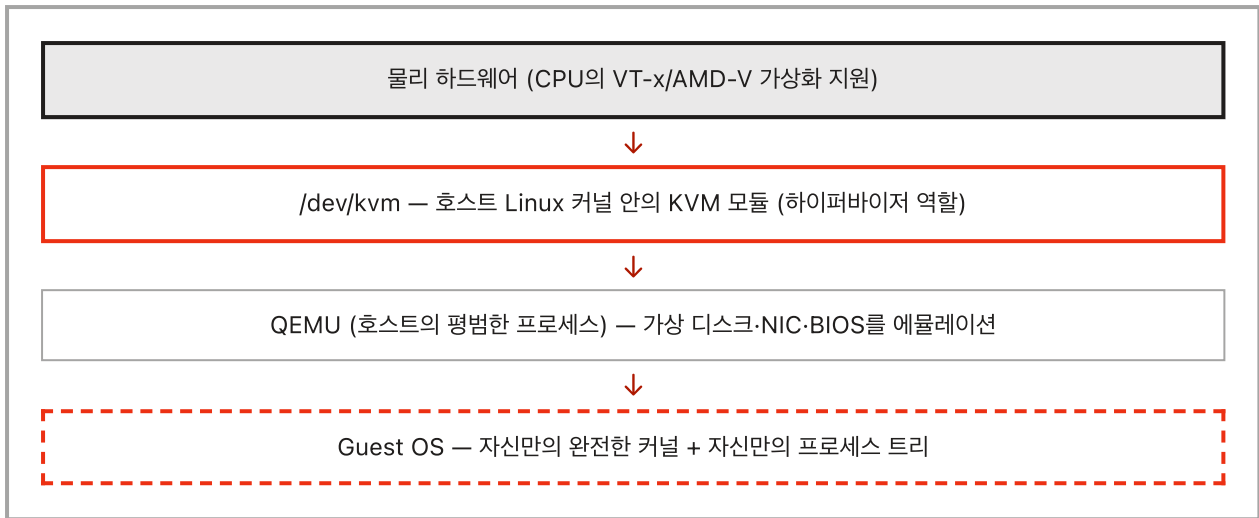
익명 파이프(pipe())는 fork로 이어진 관계에서만 쓸 수 있지만, named pipe(FIFO)는 파일시스템에 이름을 가진 노드로 존재해 서로 관계없는 두 프로세스도 연결할 수 있습니다.



둘 중 한쪽이 아직 열지 않으면 다른 쪽의 `open()`은 블록됩니다 — 양쪽이 모두 열려야 통신이 시작됩니다.

가상 머신 — KVM

컨테이너가 커널을 공유하는 것과 달리, 가상 머신은 게스트 전용의 완전한 커널을 실행합니다. KVM은 리눅스 커널 자체를 하이퍼바이저로 바꿔주는 모듈이고, 실제 디바이스 에뮬레이션은 QEMU 같은 사용자 공간 프로세스가 담당합니다.



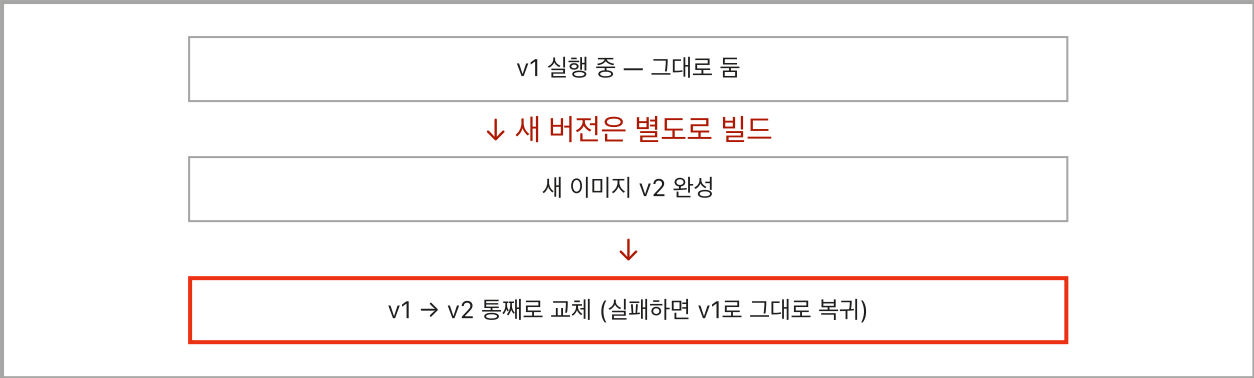
	컨테이너	가상 머신 (KVM)
커널	호스트와 공유	게스트 전용 커널 별도 실행
격리 경계	namespace + cgroup (소프트웨어)	CPU 하드웨어 가상화 (VT-x/AMD-V)
기동 속도	수백 ms 이하	수 초 (커널 부팅 필요)
공격 표면	호스트 커널을 공유 — 상대적으로 넓음	하이퍼바이저 경계로 좁게 제한

모던 리눅스 배포판

전통적인 배포판은 레고를 조각조각 조립하듯 패키지를 하나씩 설치·업데이트합니다. `apt install nginx`로 nginx만 넣고, 업데이트도 패키지 단위로 하나씩 진행됩니다 — 그래서 특정 패키지 업데이트가 다른 패키지와 충돌하거나, 업데이트 도중 정전이 나면 시스템이 일부만 바뀐 애매한 상태로 남을 수 있습니다.

모던 배포판은 시스템 전체를 사진 한 장처럼 통째로 찍어둡니다(이미지). 업데이트는 "부분 수정"이 아니라 "다음 버전 사진으로 필름 전체를 교체"하는 것과 같아서, 중간에 실패해도 이전 사진(이전 이미지)으로 완전히 되돌아갈 뿐 애매한 중간 상태가 생기지 않습니다. 컨테이너를 새 이미지로 재배포하는 것과 같은 방식을, OS 자신에게도 적용한 셈입니다.

이 원칙을 **불변성(immutability)**이라고 부릅니다: 실행 중인 시스템을 고치지 않고, 고쳐진 새 이미지를 통째로 만들어 갈아 끼웁니다. 머신이 많을수록 "하나씩 그 자리에서 패치"는 머신마다 상태가 조금씩 달라지는 위험이 커지는데, 불변성은 이를 원천적으로 막습니다.



Chrome이 백그라운드에서 새 버전을 받아두고 재시작 시 갈아 끼우는 것과 같은 원리를, OS 전체로 확장한 것입니다.

	전통적 방식	모던 방식
업데이트 단위	apt upgrade로 패키지 하나씩, 부분 실패 가능	전체 시스템 이미지를 원자적으로 교체, 실패 시 이전 이미지로 롤백
앱 배포	시스템 라이브러리에 의존	Flatpak/Snap — 앱이 필요한 라이브러리를 자기 패키지 안에 함께 담아 시스템과 분리된 채로 실행
init/서비스	배포판마다 제각각 (SysV, upstart ...)	systemd로 대부분 수렴

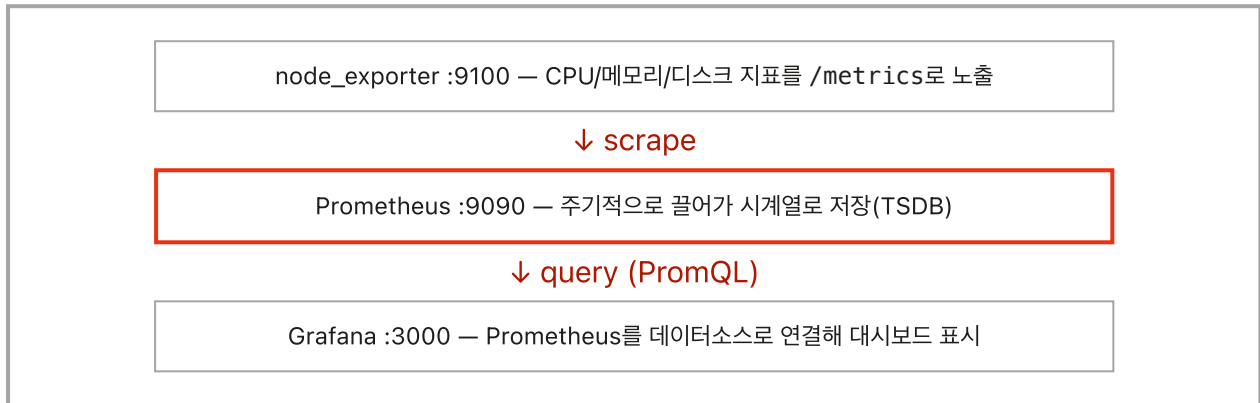
서버 쪽은 컨테이너(namespace + cgroup + OverlayFS)가 배포 단위를 대체하는 흐름이고, 데스크톱 쪽은 Flatpak/Snap이 같은 문제(의존성 충돌, 롤백 가능한 업데이트)를 앱 단위로 풀고 있습니다.

배포판	계열	주 용도
Ubuntu	Debian 계열	서버, 데스크톱
Debian	Debian 계열	서버, 안정성 중시
RHEL / CentOS Stream	RPM 계열	엔터프라이즈 서버
Fedora	RPM 계열	데스크톱, 최신 기술 실험
Amazon Linux	RPM 계열	AWS 클라우드 서버
Alpine	독립 (musl libc)	경량 컨테이너 이미지
Arch Linux	독립	데스크톱, 롤링 릴리스

핸즈온

Node Exporter → Prometheus → Grafana

로컬에서 세 컴포넌트를 띄워 "수집 → 저장 → 시각화" 파이프라인을 직접 구성합니다.



1. NODE_EXPORTER 실행

```
docker run -d --name node_exporter -p 9100:9100 prom/node-exporter
curl localhost:9100/metrics | head
```

2. PROMETHEUS 설정 & 실행

```
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: 'mymachine'
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'machine'
    static_configs:
      - targets: ['host.docker.internal:9100']
```

```
docker run -d --name prometheus -p 9090:9090 \
-v $PWD/prometheus.yml:/etc/prometheus/prometheus.yml \
prom/prometheus
```

localhost:9090/targets 에서 node exporter가 UP 상태인지 확인

3. GRAFANA 실행 & 연결

```
docker run -d --name grafana -p 3000:3000 grafana/grafana
```

localhost:3000 접속(admin/admin) → Data source에 Prometheus(http://host.docker.internal:9090) 추가 → 대시보드 > Import에서 **Node Exporter Full (ID 1860)** 불러오기

CPU에 부하를 주고(yes > /dev/null) 그래파나 대시보드에서 지표가 실시간으로 오르는 것을 확인해보세요.

프로세스 시그널 실습 (trap 포함)

셸 스크립트로 시그널을 직접 보내고 받아보며, 기본 동작과 trap으로 가로챈 동작의 차이를 확인합니다.

1. 기본 동작 — SIGTERM은 즉시 종료

```
sleep 100 &
PID=$!
kill -TERM $PID      # 기본 핸들러가 즉시 프로세스를 종료
wait $PID; echo "exit code: $?"
```

2. TRAP으로 가로채기 — GRACEFUL SHUTDOWN 흉내

```
cat > worker.sh <<'EOF'
#!/bin/bash
trap 'echo "SIGTERM 받음, 정리 중..."; sleep 2; echo "정리 완료"; exit 0' TERM

echo "작업 시작 (PID $*)"
while true; do sleep 1; done
EOF

chmod +x worker.sh
./worker.sh &                # 백그라운드로 실행
PID=$!                      # PID 저장
sleep 2
kill -TERM $PID              # 이번엔 즉시 죽지 않고 trap 블록이 먼저 실행됨
wait $PID
```

kill -TERM \$PID



trap이 등록돼 있으면 커널이 아니라 스크립트가 직접 처리



trap 블록 실행 (정리 작업) → 끝나면 exit

차이를 비교해보세요: SIGKILL(kill -9)은 trap으로도 가로챌 수 없어 정리 작업 없이 즉시 죽습니다. trap ' ' TERM처럼 빈 명령을 등록하면 해당 시그널을 완전히 무시하게 만들 수도 있습니다.

Modernist design system 기준으로 작성됨.